DriveCommand

Checklists & Workflows

Full Technical & UX Specification

Version 2.0 · 2026

Implementation-ready · Authoritative source of truth

Table of Contents

1. Product Context

DriveCommand is a multi-tenant SaaS platform for small-to-midsize trucking carriers (5–50 trucks). Users are dispatchers, owner-operators, safety managers, mechanics, and drivers — almost none of them technical. Every record is scoped by tenantId.

Stack

Layer	Technology
Monorepo	Turborepo
Web	Next.js App Router (server components, app/ directory)
Mobile	React Native 0.83 + Expo SDK 55 (apps/mobile/, EAS Build)
ORM	Prisma 7 + PostgreSQL
Auth + Storage	Supabase (migration in progress from bcryptjs)
File Storage	AWS S3
CI	GitHub Actions (typecheck, Vitest, Playwright)
Deploy	Vercel (web), EAS (mobile)

What Checklists & Workflows is: a template engine that lets carriers define reusable checklists (driver onboarding, vehicle inspections, partner setup), automatically assign them when real-world events happen, and block dispatch until required steps are complete.

What it replaces: spreadsheets, paper DVIRs, and tribal knowledge.

2. Design Philosophy

Three non-negotiable principles. Every design decision gets measured against all three.

2.1 The Midnight Owner-Op Test

A 55-year-old owner-operator with 8 trucks must be able to configure the product alone, at midnight, without calling support. If a screen requires training, the screen is wrong.

2.2 One Screen, One Action

Mobile especially: the driver has one hand free and is in a truck cab. Every screen has one primary action. Secondary actions are pushed to overflow menus or removed.

2.3 Fail Loud, Recover Easy

When something breaks (failed inspection, blocked dispatch, overdue step), the system tells the user exactly what to do next — in the notification itself, not behind three taps. No dead ends.

3. Naming Rules

Internal code uses precise technical names. UI copy uses plain English. Never mix them.

Internal (code only)	User-facing (UI copy)
Workflow Template Engine	Checklists & Workflows
Playbook	Playbook (the one exception — reads naturally)
StepTemplate	Step
Step Library	Step Library
PlaybookInstance	Active Checklist
StepInstance	Task
PlaybookTrigger	Auto-Start Rule
isDispatchBlocker	Required Before Dispatch
Skip	Skip with Reason
Tenant Admin	Account Admin

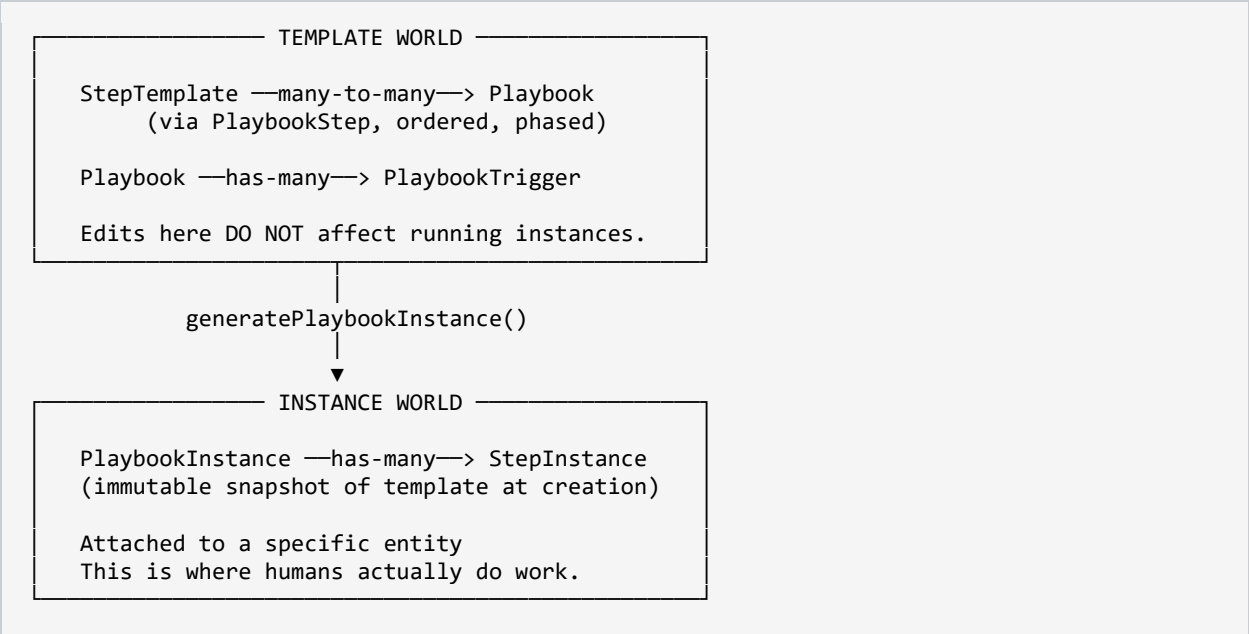
Enforcement

A Vitest test greps all .tsx files under apps/web/src/app/ and apps/mobile/src/screens/ for the words PlaybookInstance, StepInstance, StepTemplate, PlaybookTrigger in JSX text nodes and fails the build if found.

4. Architecture Overview

4.1 Two-World Model

The feature splits cleanly into template world (design time) and instance world (runtime). This is the single most important architectural decision.



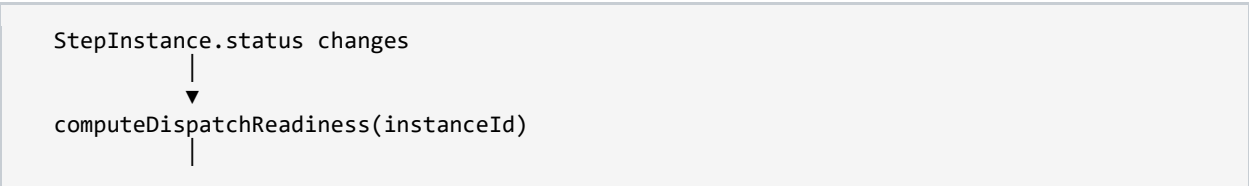
4.2 The Snapshot Rule

When a PlaybookInstance is created, the full Playbook + its Steps is deep-copied into playbookSnapshot (JSON). Each StepInstance gets its own stepSnapshot. These snapshots are never mutated.

- **Why:** an admin editing a checklist template shouldn't silently change the requirements of checklists that drivers are already halfway through. Auditors and DOT inspectors need to see exactly what was required at the time of inspection.
- **Cost awareness:** this generates ~3–10KB per instance. At 50 trucks × daily DVIRs × 365 days ≈ 55MB per tenant per year. Acceptable. If it grows past that, introduce versioned templates with a playbookVersionId pointer.

4.3 Dispatch Readiness Aggregation

Driver.isDispatchReady and Vehicle.isDispatchReady are derived columns, recomputed by computeDispatchReadiness() whenever a StepInstance changes status.



```
            ├─> updates PlaybookInstance.completionPercent
            ├─> updates PlaybookInstance.isDispatchReady
            │
            ▼
  aggregate across all active instances for entity
            │
            ▼
  updates Driver.isDispatchReady / Vehicle.isDispatchReady
            │
            ▼
  if changed → fire DISPATCH_READY notification
```

An entity is ready only when every active PlaybookInstance attached to it is ready.

4.4 Event Fan-out

Domain events (ON_DRIVER_CREATE, ON_DISPATCH_DEPART, etc.) are emitted at lifecycle boundaries of existing models. fireEvent() matches active triggers, evaluates conditions, and calls generatePlaybookInstance() for each match.

> **Condition evaluation is intentionally simple**
>
> Flat key-value equality. Example: { driverType: 'CDL' }. No expression language, no JSONLogic, no regex. This keeps the UI a dropdown instead of a formula editor. When a tenant demands richer logic, that's a future phase.

4.5 Module Boundaries

```
apps/web/src/
  app/
    checklists/
      page.tsx                    # Dashboard
      playbooks/[id]/edit/page.tsx
      playbooks/new/page.tsx
      instances/[id]/page.tsx     # Active Checklist detail
      automation/page.tsx

  server/api/routers/workflows/
    stepTemplate.ts
    playbook.ts
    instance.ts
    stepInstance.ts
    trigger.ts
    index.ts

  server/services/workflows/
    generatePlaybookInstance.ts
    computeDispatchReadiness.ts
    completeStep.ts
    failInspectionItem.ts
    fireEvent.ts
    recipes.ts
```

```
    seedStarterPlaybooks.ts

apps/mobile/src/
  screens/workflows/
    MyTasksScreen.tsx
    InspectionModeScreen.tsx
    DocumentUploadScreen.tsx
    FormFillScreen.tsx
    SignatureScreen.tsx

packages/validation/src/workflows/
  stepTemplate.ts
  playbook.ts
  instance.ts
  stepInstance.ts
  trigger.ts
```

5. Data Model

All models follow existing codebase conventions: UUID PKs, tenantId scoping, createdAt/updatedAt, soft delete via deletedAt.

5.1 Enums

```
enum StepType {
  DOCUMENT_UPLOAD  FORM_FILL  SIGNATURE  INSPECTION_ITEM
  TRAINING_ACK  APPROVAL  THIRD_PARTY  CUSTOM_NOTE
}

enum AssigneeRole { DRIVER DISPATCHER SAFETY_MANAGER ADMIN MECHANIC }

enum PlaybookCategory {
  DRIVER_ONBOARDING  VEHICLE_INSPECTION  PARTNER_ONBOARDING
  LOAD_CHECKLIST  COMPLIANCE  CUSTOM
}

enum EntityType { DRIVER  VEHICLE  DISPATCH  PARTNER  LOAD }

enum TriggerEvent {
  ON_DRIVER_CREATE  ON_VEHICLE_CREATE  ON_DISPATCH_CREATE
  ON_DISPATCH_DEPART  ON_DISPATCH_DELIVER  ON_PARTNER_CREATE
  MANUAL_ONLY  RECURRING
}

enum InstanceStatus { NOT_STARTED IN_PROGRESS COMPLETED BLOCKED }
enum StepStatus     { NOT_STARTED IN_PROGRESS COMPLETE FAILED SKIPPED }
enum PhaseType      { PRE_START DAY_1 WEEK_1 ONGOING NONE }
enum NotifType      { STEP_ASSIGNED STEP_OVERDUE INSTANCE_BLOCKED
                      DISPATCH_READY STEP_FAILED APPROVAL_NEEDED }
enum NotifChannel   { PUSH SMS IN_APP EMAIL }
```

5.2 Models

StepTemplate

Atomic reusable step definition.

```
model StepTemplate {
  id                String         @id @default(uuid())
  tenantId          String
  tenant            Tenant         @relation(fields:[tenantId],references:[id])
  name              String
  description       String?
  stepType          StepType
  assigneeRole      AssigneeRole
  requiresPhoto     Boolean        @default(false)
  requiresSignature Boolean        @default(false)
  formSchema        Json?
  documentTypeName  String?
  isActive          Boolean        @default(true)
```

```
  deletedAt         DateTime?
  createdAt         DateTime       @default(now())
  updatedAt         DateTime       @updatedAt
  playbookSteps     PlaybookStep[]
  stepInstances     StepInstance[]
  @@index([tenantId])
  @@index([tenantId, stepType])
}
```

Playbook

Named ordered collection of steps.

```
model Playbook {
  id          String              @id @default(uuid())
  tenantId    String
  tenant      Tenant              @relation(fields:[tenantId],references:[id])
  name        String
  description String?
  category    PlaybookCategory
  entityType  EntityType
  icon        String?
  color       String?
  isActive    Boolean             @default(true)
  deletedAt   DateTime?
  createdAt   DateTime            @default(now())
  updatedAt   DateTime            @updatedAt
  steps       PlaybookStep[]
  triggers    PlaybookTrigger[]
  instances   PlaybookInstance[]
  @@index([tenantId])
  @@index([tenantId, category])
  @@index([tenantId, entityType])
}
```

PlaybookStep

Join of StepTemplate to Playbook with ordering and dispatch-blocker flag.

```
model PlaybookStep {
  id                String       @id @default(uuid())
  playbookId        String
  playbook          Playbook     @relation(fields:[playbookId],references:[id])
  stepTemplateId    String
  stepTemplate      StepTemplate @relation(fields:[stepTemplateId],references:[id])
  phase             PhaseType    @default(NONE)
  sequence          Int
  isRequired        Boolean      @default(true)
  isDispatchBlocker Boolean      @default(false)
  dueDaysFromStart  Int?
  dueBeforeDispatch Boolean      @default(false)
  createdAt         DateTime     @default(now())
  updatedAt         DateTime     @updatedAt
  @@unique([playbookId, stepTemplateId, sequence])
  @@index([playbookId])
}
```

PlaybookTrigger

```
model PlaybookTrigger {
  id              String       @id @default(uuid())
  playbookId      String
  playbook        Playbook     @relation(fields:[playbookId],references:[id])
  tenantId        String
  tenant          Tenant       @relation(fields:[tenantId],references:[id])
  triggerEvent    TriggerEvent
  conditions      Json?
  recurringConfig Json?
  isActive        Boolean      @default(true)
  createdAt       DateTime     @default(now())
  updatedAt       DateTime     @updatedAt
  @@index([tenantId, triggerEvent])
}
```

PlaybookInstance

Live active checklist.

```
model PlaybookInstance {
  id                String         @id @default(uuid())
  tenantId          String
  playbookId        String
  playbook          Playbook       @relation(fields:[playbookId],references:[id])
  playbookSnapshot  Json           // immutable deep copy
  entityType        EntityType
  entityId          String
  status            InstanceStatus @default(NOT_STARTED)
  completionPercent Float          @default(0)
  isDispatchReady   Boolean        @default(false)
  startedAt         DateTime?
  completedAt       DateTime?
  dueDate           DateTime?
  createdAt         DateTime       @default(now())
  updatedAt         DateTime       @updatedAt
  stepInstances     StepInstance[]
  notifications     PlaybookNotification[]
  @@index([tenantId, entityType, entityId])
  @@index([tenantId, status])
  @@index([tenantId, isDispatchReady])
}
```

StepInstance

```
model StepInstance {
  id                String           @id @default(uuid())
  playbookInstanceId String
  playbookInstance   PlaybookInstance
@relation(fields:[playbookInstanceId],references:[id])
  stepTemplateId     String
  stepTemplate       StepTemplate     @relation(fields:[stepTemplateId],references:[id])
  stepSnapshot       Json             // immutable
  status             StepStatus       @default(NOT_STARTED)
  assigneeRole       AssigneeRole
```

```
  assignedUserId     String?
  completedByUserId  String?
  completedAt        DateTime?
  result             Json?
  skipReason         String?
  skippedByUserId    String?
  dueDate            DateTime?
  isOverdue          Boolean          @default(false)
  createdAt          DateTime         @default(now())
  updatedAt          DateTime         @updatedAt
  @@index([playbookInstanceId])
  @@index([assignedUserId, status])
  @@index([playbookInstanceId, status])
}
```

PlaybookNotification

Audit log for every notification sent.

```
model PlaybookNotification {
  id                 String           @id @default(uuid())
  tenantId           String
  playbookInstanceId String
  playbookInstance   PlaybookInstance
@relation(fields:[playbookInstanceId],references:[id])
  stepInstanceId     String?
  notificationType   NotifType
  channel            NotifChannel
  recipientUserId    String
  message            String
  sentAt             DateTime?
  deliveredAt        DateTime?
  createdAt          DateTime         @default(now())
  @@index([playbookInstanceId])
  @@index([recipientUserId])
}
```

5.3 Entity Model Updates

```
// Existing Driver — add:
isDispatchReady Boolean @default(false)

// Existing Vehicle — add:
isDispatchReady Boolean @default(false)

// Partner: no readiness flag, but gets a 'Checklists' tab.
```

6. Service Layer

Pure functions (no HTTP concerns) living in apps/web/src/server/services/workflows/. Callable from tRPC, event hooks, and cron jobs.

6.1 generatePlaybookInstance

```
async function generatePlaybookInstance(args: {
  playbookId: string;
  entityType: EntityType;
  entityId: string;
  tenantId: string;
  triggeredBy: 'manual' | 'trigger';
}): Promise<PlaybookInstance>
```

Steps

1. Load Playbook with all PlaybookSteps ordered by (phase, sequence). Throw 404 if missing, 400 if not active.
2. Deep-copy playbook + steps into playbookSnapshot.
3. Create PlaybookInstance with status=NOT_STARTED, isDispatchReady=false.
4. For each PlaybookStep: create a StepInstance with stepSnapshot, resolve assigneeRole, compute dueDate.
5. Resolve assignedUserId: query tenant users matching assigneeRole. If exactly one → assign; if multiple → null.
6. Emit STEP_ASSIGNED notification per resolvable assignee.
7. Return hydrated instance.

Error cases: Playbook 404 / not active 400 / Entity 404 / duplicate active instance for same (entity, playbook) 409.

6.2 computeDispatchReadiness

```
async function computeDispatchReadiness(instanceId: string): Promise<{
  isReady: boolean;
  blockers: StepInstance[];
}>
```

8. Load StepInstances where source PlaybookStep has isDispatchBlocker=true.
9. If any blocker is NOT_STARTED / IN_PROGRESS / FAILED → isReady=false, instance.status=BLOCKED.
10. If all blockers are COMPLETE / SKIPPED → isReady=true.
11. Recompute completionPercent = (COMPLETE + SKIPPED) / total * 100.
12. Persist.
13. If isDispatchReady flipped true → fire DISPATCH_READY to dispatcher.
14. Recompute parent entity's isDispatchReady by aggregating across all active instances.

Edge case: instance with zero blocker steps returns isReady=true, blockers=[].

6.3 completeStep

```
async function completeStep(args: {
  stepInstanceId: string;
  userId: string;
  result: StepResult;
}): Promise<StepInstance>
```

Validation by stepType — all reject with specific error codes, not generic 400:

stepType	Required in result
DOCUMENT_UPLOAD	fileUrls non-empty; template must have documentTypeName
SIGNATURE	signatureUrl is valid URL
FORM_FILL	formData passes validation against formSchema
INSPECTION_ITEM	passOrFail is 'pass' or 'fail' (fail → failInspectionItem)
TRAINING_ACK	note present OR boolean acknowledgment
APPROVAL	approver role matches assigneeRole
THIRD_PARTY	note or fileUrls
CUSTOM_NOTE	note present

Side effects

- Set status=COMPLETE, completedByUserId, completedAt.
- If DOCUMENT_UPLOAD: create document record in existing storage, labeled documentTypeName, attached to entityId.
- Call computeDispatchReadiness(instanceId).
- Emit STEP_ASSIGNED to assignee of next NOT_STARTED step.

6.4 failInspectionItem

```
async function failInspectionItem(args: {
  stepInstanceId: string;
  userId: string;
  result: { photoUrls: string[]; note?: string };
}): Promise<void>
```

15. Validate photoUrls non-empty if requiresPhoto=true.
16. Set StepInstance status=FAILED, persist result.
17. If parent Playbook category=VEHICLE_INSPECTION: create ad-hoc APPROVAL StepInstance assigned to MECHANIC titled 'Repair sign-off: [step name]', attached to same PlaybookInstance.
18. Set PlaybookInstance status=BLOCKED.
19. Emit STEP_FAILED to dispatcher + mechanic.

20. Call computeDispatchReadiness → vehicle flips not-ready.

6.5 fireEvent

```
async function fireEvent(args: {
  event: TriggerEvent;
  entityData: Record<string, any>;
  tenantId: string;
}): Promise<void>
```

21. Load active PlaybookTriggers for (tenantId, triggerEvent=event).
22. For each: evaluate conditions via flat key-value equality. Skip non-matches.
23. For each match: call generatePlaybookInstance with derived entityType and entityData.id.
24. Log to audit table.

Attachment points (wired in Phase 4)

Lifecycle	Call
Driver onCreate	fireEvent('ON_DRIVER_CREATE', driverRecord, tenantId)
Vehicle onCreate	fireEvent('ON_VEHICLE_CREATE', vehicleRecord, tenantId)
Dispatch onCreate	fireEvent('ON_DISPATCH_CREATE', dispatchRecord, tenantId)
Dispatch → DEPARTED	fireEvent('ON_DISPATCH_DEPART', dispatchRecord, tenantId)
Dispatch → DELIVERED	fireEvent('ON_DISPATCH_DELIVER', dispatchRecord, tenantId)
Partner onCreate	fireEvent('ON_PARTNER_CREATE', partnerRecord, tenantId)

7. tRPC API Surface

All procedures use existing auth middleware, Zod validation from packages/validation, and existing error-formatting conventions.

7.1 stepTemplate router

Procedure	Auth	Notes
list	tenant member	Active only; filter by stepType or assigneeRole
create	admin	Full template fields
update	admin	Cannot change stepType if instances exist
archive	admin	Soft delete via isActive=false

7.2 playbook router

Procedure	Notes
list	Returns stepCount + activeInstanceCount per playbook
get	Full playbook with ordered steps, triggers, last 5 instances
create	Empty playbook; steps added separately
update	Name/desc/icon/color only; category + entityType locked after instances exist
addStep	Inserts at sequence, re-sequences as needed
removeStep	Removes, re-sequences
reorderSteps	Batch { steps: [{playbookStepId, sequence}] }
setTrigger	Upsert; one trigger per event per playbook
removeTrigger	Hard delete trigger
duplicate	Clones playbook + PlaybookSteps; no triggers or instances
archive	Soft delete; disables all triggers

7.3 instance router

Procedure	Notes
generate	Calls generatePlaybookInstance

list	Paginated, BLOCKED first
get	Full instance + step instances + results
computeReadiness	Internal wrapper
getForEntity	All active instances for one entity — powers profile tabs

7.4 stepInstance router

Procedure	Notes
complete	Validates per type, calls completeStep
fail	INSPECTION_ITEM only, calls failInspectionItem
skip	Admin only, requires reason
requestApproval	Sets IN_PROGRESS, notifies approver
approve	Approver role, sets COMPLETE, recomputes readiness
getForDriver	Powers mobile My Tasks

7.5 trigger router

Procedure	Notes
listRecipes	Pre-built recipes + enabled/disabled per tenant
enableRecipe	Creates PlaybookTrigger from recipe
disableRecipe	Sets isActive=false
fire	Internal, called from lifecycle hooks

8. Web UX

UX baseline: Housecall Pro and Jobber. Large tap targets, one action per screen, dashboards that surface work rather than bury it.

8.1 Checklists & Workflows Dashboard — /checklists

Top — Active Work Board

Three swimlanes, only shown when at least one instance exists:

Column	Accent	Contents
Needs Attention	Red left border	BLOCKED instances + overdue steps
In Progress	Yellow left border	Active with upcoming due steps
Completed Today	Green left border	Completed in last 24h

Each card: entity name + avatar, playbook name + icon, completion ring, next/overdue step label, single action button. Sort: overdue blockers first.

Empty-state copy: 'When you start a checklist, it'll show up here.'

Middle — Your Playbooks

- Grid of cards. Filter tabs: All / Driver / Vehicle / Dispatch / Partner.
- Each card: icon, name, category badge, step count, active instance count, last used.
- 'Create New Playbook' is always the first card — dashed border, plus icon, never filtered.

Bottom — Auto-Start Rules (collapsed)

Card grid of recipe toggles. 'Custom Rules' link for power users.

8.2 Playbook Builder — /checklists/playbooks/[id]/edit

> **UX mandate**
> A dispatcher must build a functional Pre-Trip Inspection in under 10 minutes on first use.

Three-column desktop layout. Mobile collapses to stacked accordion.

Left — Playbook Details (always visible)

- Name — large editable field, auto-focused on create.
- Category — 6-tile icon grid, not dropdown.
- Entity type — auto-set by category, editable override.
- Icon — scrollable emoji grid filtered by category.
- Color — 8 preset swatches.

- Description — optional.
- Auto-Start Rules section with 'Add Rule' modal.

Auto-Start Rule Modal — 3 fields, nothing more

25. 'When does this start?' — plain-English dropdown.
26. 'Only for specific records?' — optional condition (e.g., Driver type = CDL).
27. Live preview sentence updating in real time.

Center — Canvas

- Vertical ordered step list with phase dividers: Pre-Start / Day 1 / Week 1 / Ongoing (collapsible, drag-across).
- Row: drag handle, type icon, name, assignee badge, phase tag, Required Before Dispatch toggle, due-days, delete.
- Expanded row: full instruction editable inline, photo/signature toggles, type-specific builders.
- 'Add Step' button at bottom of each phase section.

Right — Step Library

- Search. Filter chips: All / Document / Inspection / Form / Signature / Approval.
- StepTemplate cards with '+' to add. 'Create New Step' opens inline form in the panel — no separate screen.

Preview Panel (slide-in)

Toggle: Preview as Driver (phone frame) / Preview as Dispatcher (instance card). Live updates as steps added/reordered.

8.3 Active Checklist Detail — /checklists/instances/[id]

- Header card: entity name + avatar + link, playbook name + icon, status badge, completion ring, dispatch readiness banner (green 'Dispatch Ready' or red 'Blocked — N steps required'), 'Add Step' button (admin).
- Phase sections: collapsible; label + count e.g. 'Day 1 (3/5 complete)'.
- Step rows: status icon, name, assignee avatar, due date, result summary, contextual action button.

Action button by status and user role:

Status	Is Assignee	Label
NOT_STARTED	yes	Start / Upload / Fill Out / Sign Now
COMPLETE	any	View Result
FAILED	any	View Issue (red)
SKIPPED	any	Skipped — [reason] (muted)

IN_PROGRESS + APPROVAL	approver	Review & Approve

8.4 Auto-Start Rules — /checklists/automation

- Recipes: card grid — icon, name, plain-English sentence, playbook dropdown, enabled/disabled pill, 'Active X times' counter.
- Custom Rules: table Event → Condition → Playbook → Status → Edit/Delete. 'Create Custom Rule' opens a 3-step modal.

9. Mobile UX

> **Mobile mandate**
>
> Drivers use this in a cab, one hand, sometimes poor lighting. Tap targets ≥56px. Instructions ≤2 short sentences. No navigation while a task is in progress.

9.1 My Tasks — driver home tab

- Top summary: '3 of 7 tasks complete today' + thin animated progress bar. Tap expands grouped by checklist.
- Feed: vertical cards — context label, large step name, truncated instruction, due badge, full-width action button ≥56px.
- Empty state: 'You're all caught up. No open tasks right now.' — no upsell, no filler.

9.2 Inspection Mode — full-screen takeover

Triggered by 'Start Inspection' on a VEHICLE_INSPECTION instance. This is the product's signature UX.

> **Critical**
>
> Full-screen takeover. No navigation chrome. No tab bar. Mirrors Calm/Headspace focused-session UX.

Top bar

- Back arrow → confirmation 'Exit inspection? Your progress is saved.'
- Playbook name (muted center).
- Progress counter ('4 / 12') right-aligned.
- Thin progress bar below.

Step card (80% of screen)

Step number badge, step name (large bold, 2–3 words), instruction (1–2 short sentences), optional diagram.

Action area (bottom 20%)

INSPECTION_ITEM — two side-by-side buttons, ≥56px, full half-width:

PASS	FAIL
Green · slides left, next slides in	Red · expands in place to fail-capture

Fail-capture flow

- Expands in place, no navigation.

- Red 'Issue Found' header.
- Camera button — up to 3 photos, thumbnail preview.
- Notes field (optional), keyboard auto-opens.
- 'Submit & Continue' validates requiresPhoto, saves, advances.

Micro-moments

- Pass → subtle checkmark, green flash on progress bar.
- Every 3 steps: tiny encouragement ('Halfway there! 6 of 12').

Completion screen

- Full-screen success (not a toast).
- Large animated checkmark.
- 'Pre-Trip Inspection Complete' + timestamp + truck #.
- Summary: '12 passed · 0 failed' or '11 passed · 1 flagged'.
- If failures: '1 item flagged — your dispatcher has been notified.'
- Single button: 'Back to My Tasks'.

9.3 Document Upload

Full-screen. Step name large, instruction, document type label, upload area ≥200px dashed box: 'Tap to take a photo or choose from files'. Thumbnail + Replace. Submit disabled until file selected.

9.4 Form Fill

Full-screen form, no modals. Fields rendered from formSchema:

- Boolean → large YES/NO toggle buttons (not checkboxes).
- Date → native date picker.
- Select → bottom sheet picker (not dropdown).
- Text → large input, auto-scroll on keyboard.

Submit validates required fields with inline error per field on blur.

9.5 Signature

Full-screen pad. Instruction, signature canvas (full width, ≥200px), 'Clear' top right, 'I confirm and sign' submits with timestamp + userId.

10. Notifications

Copy rule

Every notification contains the action, not just an alert. Recipient should know what to do from the notification alone.

Type	Recipient + Channel	Message
STEP_ASSIGNED	Driver — Push + SMS	Push: '[Company]: New task ready — [Step Name]. Tap to complete.' SMS includes deep link.
STEP_OVERDUE	Dispatcher — Push (24h after due)	'[Driver] hasn't completed [Step] — due X days ago. Tap to send a reminder.'
INSTANCE_BLOCKED	Dispatcher — Push (immediate)	'[Driver] is blocked from dispatch — [Step] is required. Tap to review.'
DISPATCH_READY	Dispatcher — Push (on flip)	'[Driver] is dispatch ready — all required steps complete.'
STEP_FAILED	Dispatcher — Push	'[Driver] flagged an issue on Truck #[X]: [Step]. Photo attached. Tap to review.'
STEP_FAILED	Mechanic — Push	'Repair needed on Truck #[X]: [Step] flagged by [Driver]. Tap to sign off.'
APPROVAL_NEEDED	Approver — Push	'[Name] completed [Step] and needs your approval. Tap to review.'

Channel logic by role

- Drivers → Push + SMS. SMS always sent for STEP_ASSIGNED.
- Dispatchers → Push only.
- Safety Managers → Push + daily email digest.
- Admins → In-app for non-critical. Email INSTANCE_BLOCKED if instance >48h old.
- Mechanics → Push for repair sign-offs.

11. Automation Recipes

Stored as configuration constants in apps/web/src/server/services/workflows/recipes.ts. Seeded to PlaybookTrigger on enable.

Recipe Key	Display Name	Trigger + Condition
cdl_driver_onboarding	Start CDL Driver Onboarding automatically	ON_DRIVER_CREATE · { driverType: 'CDL' }

non_cdl_driver_onboarding	Start Non-CDL Driver Onboarding automatically	ON_DRIVER_CREATE · { driverType: 'NON_CDL' }
owner_op_onboarding	Start Owner-Operator Onboarding automatically	ON_DRIVER_CREATE · { driverType: 'OWNER_OP' }
pre_trip_inspection	Assign Pre-Trip Inspection on every dispatch	ON_DISPATCH_CREATE · no condition
post_trip_inspection	Assign Post-Trip Inspection after every delivery	ON_DISPATCH_DELIVER · no condition
new_vehicle_intake	New truck intake checklist	ON_VEHICLE_CREATE · no condition
partner_onboarding	Start partner onboarding when a new broker is added	ON_PARTNER_CREATE · no condition

12. Starter Seed Data

Three Playbooks auto-created for new tenants on signup. Fully editable.

Starter 1 — CDL Driver Onboarding

Category: DRIVER_ONBOARDING · Entity: DRIVER

#	Step	Type · Assignee · Blocker · Due
1	Upload Driver's License	DOCUMENT_UPLOAD · ADMIN · Blocker · Day 0
2	Upload Medical Certificate	DOCUMENT_UPLOAD · ADMIN · Blocker · Day 0
3	Pre-Employment Drug Test	THIRD_PARTY · SAFETY_MANAGER · Blocker · Day 0
4	Driver Application Form	FORM_FILL · DRIVER · Blocker · Day 0
5	FMCSA Clearinghouse Query	THIRD_PARTY · SAFETY_MANAGER · Blocker · Within 3 days
6	Motor Vehicle Record (MVR)	THIRD_PARTY · SAFETY_MANAGER · Blocker · Within 3 days
7	Safety Policy Acknowledgment	TRAINING_ACK · DRIVER · Not Blocker · Day 1
8	ELD Training Completion	TRAINING_ACK · DRIVER · Not Blocker · Week 1
9	Driver Signature	SIGNATURE · DRIVER · Blocker · Day 0

Starter 2 — Pre-Trip Inspection (DVIR)

Category: VEHICLE_INSPECTION · Entity: DISPATCH · All steps: INSPECTION_ITEM · DRIVER · Blocker · requiresPhoto on FAIL.

#	Step	Instruction
1	Front Brakes	Press pedal firmly. Check for resistance and unusual sounds.
2	Rear Brakes	Check brake lines for leaks. Test parking brake.
3	Tires & Wheels	Check tread depth, inflation, and sidewall condition on all tires.
4	Lights	Verify headlights, brake lights, turn signals, and clearance lamps.
5	Mirrors	Confirm all mirrors are clean, undamaged, and properly adjusted.
6	Windshield & Wipers	Check for cracks. Test wiper operation and washer fluid.
7	Horn	Test horn operation.
8	Fuel Level	Confirm adequate fuel for the route.

9	Engine Compartment	Check oil, coolant, belts, and hoses for leaks or damage.
10	Coupling Devices	Inspect fifth wheel or hitch. Confirm trailer connection if applicable.
11	Emergency Equipment	Confirm reflectors, fire extinguisher, and first aid kit are present.
12	Driver Signature	SIGNATURE · DRIVER · Blocker — confirms inspection completed.

Starter 3 — New Partner Setup (Carrier Packet)

Category: PARTNER_ONBOARDING · Entity: PARTNER

#	Step	Type · Assignee · Blocker
1	Upload W-9	DOCUMENT_UPLOAD · ADMIN · Blocker
2	Upload Certificate of Insurance	DOCUMENT_UPLOAD · ADMIN · Blocker
3	Upload Letter of Authority	DOCUMENT_UPLOAD · ADMIN · Blocker
4	Broker-Carrier Agreement	SIGNATURE · ADMIN · Blocker
5	Payment Terms Confirmation	FORM_FILL · ADMIN · Not Blocker
6	Partner Approval	APPROVAL · ADMIN · Blocker

13. Integration Points

Module	Required Change
Driver model	Add isDispatchReady Boolean. Aggregated by computeDispatchReadiness. Surface on profile + load-assignment screen with badge.
Vehicle model	Add isDispatchReady Boolean. Same aggregation pattern for DVIR blockers.
Dispatch creation	Before save: check driver + vehicle readiness. If false, modal 'This driver has incomplete required steps. View checklist or override.' Override requires admin + logged reason.
Load assignment screen	Surface readiness badges. Green check = ready. Red = blocked with open-blocker count.
Driver profile	New 'Checklists' tab showing all PlaybookInstances with status, %, link to detail.
Vehicle profile	Same pattern.
Partner profile	Same pattern.
Driver onCreate	Attach fireEvent('ON_DRIVER_CREATE', ...) (Phase 4).
Vehicle onCreate	Attach fireEvent('ON_VEHICLE_CREATE', ...) (Phase 4).
Dispatch status	Hooks on transitions to DEPARTED and DELIVERED (Phase 4).
Partner onCreate	Attach fireEvent('ON_PARTNER_CREATE', ...) (Phase 4).
Document storage	DOCUMENT_UPLOAD completion uses existing Supabase/S3 pipeline. Creates document record labeled with documentTypeName.
Notification service	Map NotifChannel to existing push/SMS/email/in-app methods. No new providers.
Mobile nav	Add 'My Tasks' tab to driver bottom tab navigator. Badge = open DRIVER-role steps count.

14. Phased Build Plan

Philosophy: ship value incrementally. Each phase independently useful. Each ends on green CI, a manual demo of its Definition of Done, and a clean commit on master.

Phase 1 — Foundation

Definition of Done: Admin creates a Playbook, adds steps from library, saves. No runtime.

- Prisma: StepTemplate, Playbook, PlaybookStep migrations.
- tRPC: stepTemplate CRUD, playbook CRUD, step management.
- Web: Playbook Builder (left + center + right columns).
- Web: Playbook card grid on /checklists (grid only, no Work Board).
- Seed: 3 starter Playbooks on tenant create.
- Excluded: triggers, instances, mobile.

Phase 2 — Execution

Definition of Done: Dispatcher creates an Active Checklist manually. Driver completes non-inspection steps on mobile.

- Prisma: PlaybookInstance, StepInstance migrations.
- tRPC: instance.generate/list/get/getForEntity/computeReadiness, stepInstance.complete/skip/getForDriver.
- Services: generatePlaybookInstance, computeDispatchReadiness, completeStep.
- Web: Active Checklist Detail screen.
- Web: 'Checklists' tab on Driver/Vehicle/Partner profiles.
- Web: Active Work Board swimlanes (manual instances only).
- Mobile: My Tasks tab.
- Mobile: Document Upload, Form Fill, Signature screens.
- isDispatchReady surfaced on driver profile — NOT enforced yet.
- fireEvent hook points marked with TODOs, not wired.

Phase 3 — Inspection Mode

Definition of Done: Driver executes full DVIR via card-by-card Inspection Mode. Fails auto-create mechanic sign-offs. Vehicle readiness enforced.

- Mobile: full-screen Inspection Mode UX (pass/fail, fail capture, completion, micro-moments).
- Service: failInspectionItem.
- tRPC: stepInstance.fail, .requestApproval, .approve.
- Notifications: STEP_FAILED, APPROVAL_NEEDED.
- Vehicle.isDispatchReady computed and surfaced.
- Notification infra: push + SMS for STEP_ASSIGNED + STEP_FAILED.

Phase 4 — Automation

Definition of Done: Playbooks fire automatically. Tenants toggle recipes. Dispatch blocked for non-ready drivers.

- Prisma: PlaybookTrigger migration.
- tRPC: trigger router.
- Service: fireEvent (flat key-value conditions only).
- Hooks wired on Driver/Vehicle/Dispatch/Partner lifecycle.
- Web: Auto-Start Rules page (recipes + custom rules).
- Recipe library constants (all 7).
- Dispatch enforcement with admin override + audit log.
- Full notification suite.

Phase 5 — Polish & Analytics

Definition of Done: Preview panel live. Analytics on dashboard. SMS confirmed in staging.

- Web: Preview Panel slide-in (phone frame + dispatcher card).
- SMS verified end-to-end.
- Analytics: completion rate, average time, drop-off.
- Daily email digest for Safety Managers.
- Overdue alerts 24h after due date.
- Skip-with-reason audit trail visible on instance detail.
- Tenant automation activity log.

15. Testing Strategy

Layers in order of increasing cost:

28. TypeScript — strict mode; tsc --noEmit per package in CI.
29. Unit (Vitest) — services as pure functions.
30. Integration (Vitest + test DB) — tRPC procedures end-to-end with seeded DB.
31. E2E Web (Playwright) — critical paths (builder, instance completion, dispatch block).
32. E2E Mobile (Expo + Detox/Maestro) — Inspection Mode pass/fail path.

Per-phase coverage gates

Phase 1

- Typecheck green in apps/web + packages/validation.
- Unit: tenant scoping, soft-delete behavior, step re-sequence edge cases.
- Integration: full CRUD round-trip for playbook + steps.
- Seed test: new tenant gets exactly 3 starter Playbooks.
- Naming lint: grep fails build if internal names appear in .tsx text nodes.

Phase 2

- Snapshot immutability: generate, mutate source, assert snapshot unchanged.
- Readiness with zero blockers returns isReady=true.
- completeStep type-specific validation — one test per StepType, error code asserted.
- Mobile tap-target audit: fails if any Pressable/TouchableOpacity has height < 56.

Phase 3

- failInspectionItem creates exactly one mechanic APPROVAL step when category=VEHICLE_INSPECTION.
- Readiness flips false after fail, true after mechanic approval.
- Photos rejected if requiresPhoto=true and photoUrls empty.
- E2E: complete 12-step DVIR with one intentional fail, verify completion copy.

Phase 4

- fireEvent matches condition, skips mismatch.
- Disabling a recipe stops future spawns, leaves existing instances untouched.
- Override audit record written with reason + userId.
- Dispatch creation blocked for non-ready driver without override.

Phase 5

- Preview panel renders both views, updates on step reorder.
- SMS to real staging number confirmed delivered.
- Overdue alert fires exactly at 24h (time-mocked).

- Analytics numbers match raw SQL aggregates.

16. Implementation Instructions for Claude Code

This section is written for the AI agent, not humans.

16.1 One-Time Setup

33. This document lives at docs/specs/workflow-engine.md.
34. Reference line appended to root CLAUDE.md.
35. Feature branch per phase: feat/workflow-phase-1-foundation, etc.
36. One PR per phase.

16.2 Per-Phase Workflow (GSD)

Step	GSD Command	Purpose
1	/gsd:discuss-phase	Read spec, read codebase, surface gaps & questions
2	/gsd:plan-phase	Produce .gsd/phase-N-plan.md with atomic tasks
3	/gsd:execute-phase	Work tasks one at a time, commit per task
4	/gsd:verify-work	Run verification checks against Section 14 DoD

Do not skip steps. Do not batch phases. If execute reveals a plan gap, stop and go back to plan — do not improvise scope.

16.3 Hard Rules

- **Spec is truth.** If codebase contradicts spec on something spec is explicit about, follow spec and flag in PR.
- **Read the codebase before writing code.** Existing patterns already exist. Do not invent alternatives.
- **No mid-phase scope creep.** Write related improvements to docs/tech-debt.md, don't build them.
- **Tenant scoping mandatory.** Every query filters by tenantId. Phase 1 verification greps for this.
- **User-facing names only in UI.** Internal names never appear in .tsx rendered text. Lint-enforced.
- **Tap targets ≥56px on mobile.** Audit-enforced.
- **Snapshots immutable.** Tested in Phase 2.
- **Use UI UX Pro Max skill** per CLAUDE.md for every UI task.

16.4 File-Writing Conventions

- New Zod schemas: packages/validation/src/workflows/ if pattern exists; otherwise co-locate with router.
- New services: apps/web/src/server/services/workflows/.

- New routers: apps/web/src/server/api/routers/workflows/.
- Mobile screens: apps/mobile/src/screens/workflows/.
- Prisma additions: preserve alphabetical ordering if existing schema does; otherwise append.

16.5 Definition of Done Verification Template

```
Phase N — Verification Report

DoD checks:
  [✓] 1. <check description> — evidence
  [✗] 2. <check description> — failure detail + proposed fix
  ...

Guardrails:
  [✓] typecheck (apps/web)
  [✓] typecheck (packages/validation)
  [✓] vitest (apps/web)
  [✓] playwright (if applicable)
  [✓] naming-lint
  [✓] tap-target-audit (if applicable)
  [✓] tenant-scoping-grep

Tech debt noted: <list or 'none'>

Merge decision: ✓ ready / ✗ blocked
```

17. Prompts (Copy-Paste Ready)

Run these in order. Wait for each to fully complete and verify before the next. All prompts assume this document is at docs/specs/workflow-engine.md (Prompt 0 gets it there).

Prompt 0 — One-Time Setup

```
Use the GSD skill with /gsd:quick.

Task: Install the Workflow Engine specification into the repository.

Steps:
1. Create docs/specs/ at the repo root if missing.
2. I will attach two files in a follow-up: workflow-engine.md (the full spec) and
   DriveCommand_Workflow_Engine_Technical_Spec.docx (human-readable mirror).
   Save both to docs/specs/ unchanged.
3. Append the following block to the existing root CLAUDE.md
   (do not overwrite existing content):

# Workflow Engine Spec — Always Load

When any task touches the "Checklists & Workflows" feature, Playbooks, Step
Templates, Active Checklists, Tasks, Auto-Start Rules, or DVIR flows:

1. Read docs/specs/workflow-engine.md in full before writing code.
2. Section 14 defines scope per phase — do not build ahead.
3. UI copy uses only user-facing names (Section 3 naming table).
4. Follow existing codebase conventions for Prisma, tRPC, auth, upload,
   and notifications — do not introduce new patterns.

4. Create branch feat/workflow-engine-spec and commit:
   "docs: add workflow engine spec and loader directive".

Constraints:
- Do not modify any other files.
- Do not run any code generation.
- Do not install dependencies.

Output: confirmation, CLAUDE.md diff, commit SHA.
```

Prompt 1 — Phase 1: Discuss

```
Use the GSD skill with /gsd:discuss-phase.

Context: Building "Checklists & Workflows" for DriveCommand. Full spec at
docs/specs/workflow-engine.md. Read it in full before answering.

Scope for this discussion: Phase 1 — Foundation (Section 14).
Template creation only. No runtime, no triggers, no mobile.
```

```
Before responding, analyze:
- prisma/schema.prisma (conventions, tenantId scoping, soft delete, indices)
- apps/web/src/server/api/routers/ (auth middleware, Zod patterns)
- apps/web/src/app/ (App Router conventions, layout structure)
- packages/validation (shared Zod schemas if present)
- Existing new-tenant onboarding/seed logic

Return:
1. Three-sentence restatement of Phase 1 scope.
2. Gaps between spec and codebase — name specific files and patterns.
3. Proposed file structure for new work (routers, components, services, seed).
4. Open questions, max 5, ranked by blocker severity.
5. Risks for the three-column Playbook Builder given existing UI components.

Constraints:
- No code.
- No Phase 2+ work.
- UI copy in examples uses only user-facing names (Section 3).
- Stack per CLAUDE.md: Next.js + Tailwind + shadcn/ui.
```

Prompt 2 — Phase 1: Plan

```
Use the GSD skill with /gsd:plan-phase.

Context: Phase 1 discussion complete. My answers are in this thread.
Produce the execution plan for Phase 1 — Foundation.
Spec: docs/specs/workflow-engine.md, Section 14 Phase 1 only.

Plan must include:
1. Ordered atomic tasks, each independently verifiable.
2. Per task: files to create/modify, files NOT to touch, expected lines.
3. Single Prisma migration add_workflow_templates covering StepTemplate,
   Playbook, PlaybookStep only.
4. Separate tRPC router task per router (stepTemplate, playbook).
5. UI: Playbook Builder (Section 8.2) and Playbook card grid
   (Section 8.1 middle only). No Work Board, no Auto-Start Rules page.
6. Seed task for the 3 starter Playbooks (Section 12).
7. Per-layer test coverage task.
8. Verification criteria mapping to Phase 1 DoD in Section 14.

Constraints:
- Apply UI UX Pro Max per CLAUDE.md. Run:
  python3 .claude/skills/ui-ux-pro-max/scripts/search.py \
    "playbook builder three column" --design-system -p "DriveCommand"
- No work beyond Phase 1.
- Flag any change to existing Driver/Vehicle/Dispatch models — Phase 2+.

Output: .gsd/phase-1-plan.md
```

Prompt 3 — Phase 1: Execute

```
Use the GSD skill with /gsd:execute-phase.

Execute .gsd/phase-1-plan.md task by task. After each task:
1. Run npx tsc --noEmit in the affected workspace.
2. Run relevant Vitest suites.
3. Commit with a conventional commit message.
4. Pause and report before starting the next task.

Constraints:
- If a task reveals a plan gap, stop and report — do not improvise.
- User-facing names only in UI copy.
- Existing soft-delete pattern (deletedAt DateTime?).
- Existing tRPC auth middleware — no new auth patterns.
- Zod schemas in packages/validation if pattern exists; else co-locate.
- Playbook Builder UI passes UI UX Pro Max Pre-Delivery Checklist.
- Do not modify any file outside the plan.

Output: commit per task, final summary of every file changed, test output.
```

Prompt 4 — Phase 1: Verify

```
Use the GSD skill with /gsd:verify-work.

Verify Phase 1 against the DoD in Section 14 of
docs/specs/workflow-engine.md.

Required checks:
1. Admin creates a Playbook end-to-end through the UI.
2. Step Library loads; inline Step creation works.
3. Steps add, reorder (drag-and-drop), and remove on the canvas.
4. Phases (Pre-Start / Day 1 / Week 1 / Ongoing) render as dividers;
   steps move between them.
5. /checklists renders the Playbook card grid with filter tabs.
6. Brand-new tenant gets exactly 3 starter Playbooks seeded.
7. Every Prisma query is tenantId-scoped — grep and report any miss.
8. npx tsc --noEmit passes in apps/web and packages/validation.
9. npx vitest run passes in apps/web.
10. No internal names (PlaybookInstance/StepInstance/StepTemplate/
    PlaybookTrigger) in user-visible .tsx text.

Report format: Section 16.5 template.

Do not mark Phase 1 complete until all 10 pass.
```

Prompt 5 — Phase 2: Discuss

```
Use the GSD skill with /gsd:discuss-phase.

Context: Phase 1 verified and merged. Begin Phase 2 — Execution.
Spec sections to read in full first: 6 (Service Layer), 8.3 (Active
```

```
Checklist Detail), 9.1/9.3/9.4/9.5 (mobile non-inspection), 14 Phase 2.

Scope: manual instance creation, step completion (non-inspection),
mobile My Tasks, Document Upload / Form Fill / Signature screens,
Active Checklist Detail, Active Work Board swimlanes on dashboard,
readiness flag surfaced but NOT enforced.

Analyze before proposing:
- Existing document upload / S3 flow.
- Existing notification service.
- apps/mobile navigator structure.
- Existing Driver/Vehicle/Partner profile pages.

Return:
1. Three-sentence scope restatement.
2. Integration points for fireEvent() to be wired in Phase 4.
3. Snapshot strategy confirmation.
4. Proposed file structure for services.
5. Open questions, max 5.

Constraints:
- No code.
- No Phase 3+ work.
- Do not enforce dispatch blocking — surface flag only.
```

Prompt 6 — Phase 2: Plan

```
Use the GSD skill with /gsd:plan-phase.

Build the Phase 2 — Execution plan. Spec: Section 14 Phase 2.

Include:
1. Migration add_workflow_instances for PlaybookInstance + StepInstance only.
2. Service files with signatures matching Section 6 exactly.
3. tRPC per Section 7.3 and 7.4, Phase 2 subset.
4. Web: Active Checklist Detail, Active Work Board, Checklists tab on profiles.
5. Mobile: My Tasks, Document Upload, Form Fill, Signature.
   NO Inspection Mode.
6. Tests: snapshot immutability, readiness, completeStep validation-by-type.
7. Explicit NON-scope list.

Constraints:
- Run UI UX Pro Max search for "active work board swimlanes dashboard".
- Run UI UX Pro Max search for "mobile task feed one handed driver".
- Mobile tap targets ≥56px.
- Use existing upload and notification services.

Output: .gsd/phase-2-plan.md
```

Prompt 7 — Phase 2: Execute

```
Use the GSD skill with /gsd:execute-phase.

Execute .gsd/phase-2-plan.md task by task. Typecheck, test, commit,
pause, report after each.

Constraints:
- Snapshot deep-copied at generation — test mutates source Playbook
  and asserts snapshot unchanged.
- computeDispatchReadiness returns isReady=true, blockers=[] when
  no blockers exist.
- completeStep rejects with type-specific error codes.
- Mobile screens pass 56px tap target audit.
- Do not wire fireEvent() hooks — leave TODOs referencing Phase 4.
- Do not enforce dispatch blocking — surface readiness badge only.

Output: commit per task, change summary, full test output.
```

Prompt 8 — Phase 2: Verify

```
Use the GSD skill with /gsd:verify-work.

Verify Phase 2 against Section 14 Phase 2 DoD.

Checks:
1. Dispatcher creates an Active Checklist manually.
2. Active Checklist Detail renders phase sections, step rows,
   contextual action buttons.
3. Driver opens mobile → My Tasks shows open DRIVER steps.
4. Document Upload — file uploads via existing S3/Supabase,
   document record created, status flips to COMPLETE.
5. Form Fill — formData validated against formSchema.
6. Signature — persisted with timestamp and userId.
7. Active Work Board — three swimlanes, BLOCKED-first sort.
8. Driver/Vehicle/Partner profiles have a Checklists tab.
9. Dispatch readiness surfaces on driver profile but does NOT block.
10. Snapshot immutability test passes.
11. Typecheck + Vitest pass.
12. No internal names in UI copy.

Report per Section 16.5 template. Do not proceed until all pass.
```

Prompt 9 — Phase 3: Discuss

```
Use the GSD skill with /gsd:discuss-phase.

Context: Phase 2 verified and merged. Begin Phase 3 — Inspection Mode.
Spec sections: 9.2 (Inspection Mode UX), 6.4 (failInspectionItem),
14 Phase 3.

Scope: full-screen mobile Inspection Mode, failInspectionItem with
```

```
auto-creation of mechanic approval, Vehicle readiness enforcement,
STEP_FAILED and APPROVAL_NEEDED notifications.

Analyze before proposing:
- React Navigation in apps/mobile — how to remove chrome.
- Existing camera/photo capture module.
- Existing notification provider wiring.
- Vehicle model location for isDispatchReady addition.

Return:
1. Three-sentence scope restatement.
2. Full-screen takeover approach — pick one and justify.
3. Card transition animation approach — use existing dependency.
4. failInspectionItem interaction order with computeDispatchReadiness.
5. Open questions, max 5.

Constraints:
- No new animation libraries.
- No Phase 4 work.
- Tap targets ≥56px.
```

Prompt 10 — Phase 3: Plan + Execute + Verify

```
Use the GSD skill. Run /gsd:plan-phase → /gsd:execute-phase →
/gsd:verify-work sequentially for Phase 3.

Plan includes:
1. Full-screen Inspection Mode navigator/screen.
2. Card transition (pass slides left, next slides in right).
3. Fail capture — in-place expansion, up to 3 photos, notes,
   requiresPhoto enforcement.
4. Between-step micro-moments.
5. Final completion screen — full-screen success, pass/fail counts.
6. Service failInspectionItem per Section 6.4.
7. tRPC: stepInstance.fail, .requestApproval, .approve.
8. Notifications: STEP_FAILED (dispatcher + mechanic), APPROVAL_NEEDED.
9. Vehicle.isDispatchReady computed + surfaced.
10. Tests: fail creates mechanic step, readiness flips, photos enforced.

Execute constraints:
- Run UI UX Pro Max for "mobile inspection flow focused session".
- Pass/fail buttons ≥56px, full half-width.
- Exit confirmation on back arrow.
- No tab bar during inspection — screenshot test.

Verify checks:
1. Driver completes 12-step DVIR full-screen — no chrome.
2. Intentional fail on step 4 expands card, enforces photo, advances.
3. Completion screen shows correct counts.
4. Failed step creates one mechanic approval step.
5. Vehicle readiness flips correctly.
6. STEP_FAILED fires to dispatcher AND mechanic.
7. APPROVAL_NEEDED fires to mechanic.
8. Typecheck + Vitest + E2E mobile pass.
```

```
Constraints: no Phase 4 work; no dispatch block in load assignment.

Output: phase-3-plan.md, commits, change summary, verification report.
```

Prompt 11 — Phase 4: Discuss + Plan + Execute + Verify

```
Use the GSD skill. Run the full sequence for Phase 4 — Automation.

Spec: Sections 7.5, 6.5, 8.4, 11, 13, 14 Phase 4.

Scope:
1. Migration add_workflow_triggers for PlaybookTrigger.
2. tRPC trigger router per Section 7.5.
3. fireEvent service per Section 6.5 — flat key-value only.
4. Wire fireEvent into Driver/Vehicle/Dispatch/Partner lifecycle.
   Replace Phase 2 TODOs.
5. Recipe library constants (all 7 per Section 11).
6. Web: Auto-Start Rules screen.
7. Dispatch enforcement: load assignment blocks non-ready drivers
   with admin override + logged reason.
8. Full notification suite.

Constraints:
- Conditions: flat key-value only. No JSONLogic, no regex, no expressions.
- fireEvent completes before triggering transaction commits.
- Admin override MUST log reason + userId to audit table.
- Phase 2 manual instance creation must still work.
- Run UI UX Pro Max for "recipe toggle cards automation rules".

Verify checks:
1. CDL driver creation with recipe enabled spawns instance.
2. Non-CDL driver with only CDL recipe enabled does NOT spawn.
3. Toggling a recipe off stops future spawns.
4. Dispatch for non-ready driver shows override modal.
5. Admin override logs to audit table with reason.
6. All 7 recipes on /checklists/automation with plain-English copy.
7. fireEvent test: condition mismatch skips, match triggers.
8. Typecheck + Vitest + Playwright pass.

Output: phase-4-plan.md, commits, verification report.
```

Prompt 12 — Phase 5: Discuss + Plan + Execute + Verify

```
Use the GSD skill. Run the full sequence for Phase 5 — Polish & Analytics.

Spec: Section 8.2 Preview Panel, Section 10 SMS, Section 14 Phase 5.

Scope:
1. Preview Panel slide-in — phone-frame + dispatcher card, live updates.
```

```
2. SMS delivery via existing provider — retry + delivery-status log.
3. Analytics: completion rate, average time, drop-off — admin dashboard.
4. Daily email digest for Safety Managers.
5. Overdue alerts — scheduled job 24h after due date.
6. Skip with Reason — admin only, reason required, audit trail.
7. Tenant automation activity log.

Constraints:
- Use existing job scheduler.
- Preview panel read-only — no navigation or mutation.
- Analytics queries tenantId-scoped, DB aggregation.
- Run UI UX Pro Max for "playbook builder preview phone frame"
  and "analytics dashboard completion rates".

Verify:
1. Preview panel toggles views, live step updates.
2. Test SMS sends to a real staging number.
3. Analytics numbers match raw DB aggregates.
4. Overdue alert fires at exactly 24h (time-mocked).
5. Skip-with-reason audit immutable + visible.
6. Typecheck + full suite pass.

Output: phase-5-plan.md, commits, verification report,
release-notes.md summarizing Phases 1–5.
```

Quick-Prompt (Scope Reset Mid-Phase)

```
Use GSD. Spec: docs/specs/workflow-engine.md Section 14 Phase [N].
Build only what that phase's DoD lists. Do not touch future phases.
User-facing names only (Section 3 naming table). Run UI UX Pro Max
for any design work per CLAUDE.md. Commit per task. Stop and report
after each task.
```

— End of Document — v2.0 —